UFR de mathématique et d'informatique
Université de Strasbourg

Runestones Devs' Gem Hunter

Requirements Document

Ahmet Berk Ates, Ugo Berton, Solène Besancon, Anna Callet, Louise-Anne Corbé, Hassan El Sahily, Maxime Zingraff

Integrator Project – University of Strasbourg, 2025

Project management: Maxime Zingraff

Supervisor: Jason Golda, MSc

Table of Contents

I. Description of the Game 2
A) Rules and Gameplay 2
B) Key Elements 5
II. Technologies 7
III. Role of the Project Manager 9
IV. Action plan 11
A) Development Teams 11
B) Project Tasks 13
C) Gantt Chart 15

I. Description of the Game

A) Rules and Gameplay

The game we are going to develop is an adaptation of two retro games: **Wii Sports Shooting Range** and **Pokemon Snap**.

Wii Sports Shooting Range is a mini-game included in Wii Play and released in 2006. It's inspired by Duck Hunt and involves shooting at moving targets to score points. There are different types of targets, and you can play with several Wiimote controllers. However, it's a 2D experience only.

That's why we wanted to combine it with Pokemon Snap, a 3D game released on Nintendo 64 in 1999. The player explores an island populated by Pokémon in an automatic vehicle, and has to take photos of the pokémons.



Figure 1: Screenshot from Pokémon Snap, ©Nintendo



Figure 2: Screenshot from Wii Sports Shooting Range, ©Nintendo

Our game, entitled **'Gem Hunter'**, is inspired by this. Players joining the lobby will be divided into **groups of 4 players** in different games, and will have to **shoot at different targets** to **collect as many points as possible** until the end of the game. It will take place in **a mine, in a cartoon format**. The game is played in a **series of short games** with different opponents, or with the same group of friends if you wish.



Figure 3: Example of a similar visual identity, ©iStock

Here's a typical game session:

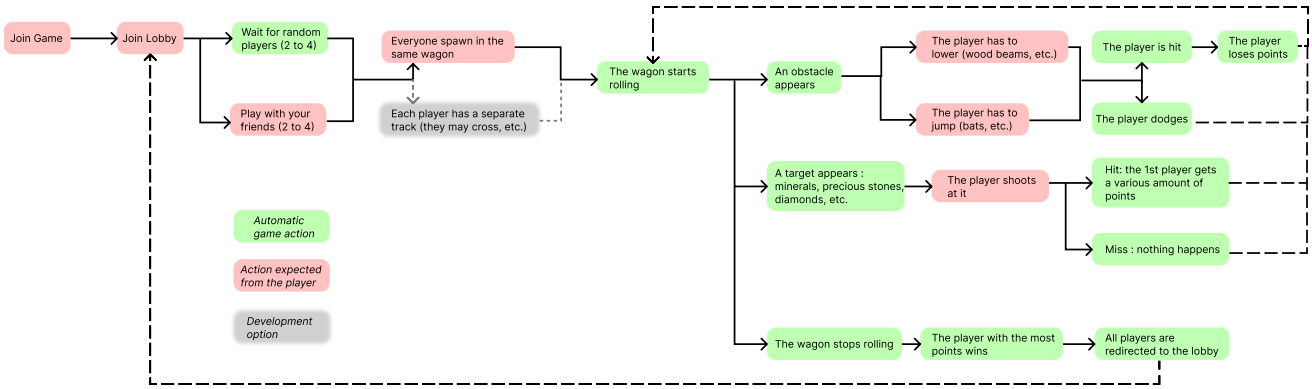


Figure 4: Game session algorithm

The shooting mechanics have not yet been defined, but various options are being considered:

- On click, the intended target is **instantly destroyed** (as in Pokemon Snap and Wii Sports Shooting Range).
- Players are **equipped with weapons**, which send a projectile towards the target, with a delay to allow the projectile to reach the target. If this option is chosen, it is possible to vary the type of weapon and therefore its characteristics (greater speed, larger projectiles, higher rate of fire, etc.).

As far as player movement is concerned, they will of course be able to **move the camera**; they can also **crouch and maybe jump** to avoid obstacles coming towards them.

Depending on the development option chosen (shown in grey on the diagram), they will also be able to move inside the carriage or not.

- If the players are **all in the same carriage**, then it's a good idea for them to move around (to change sides, avoid obstacles, etc.). In the event of a collision, the other players will be able to switch to ghost mode to avoid interfering too much with the gameplay.
- If each player **has their own carriage**, then it's not necessarily interesting for them to move around inside it (as in Pokemon Snap).

Regarding **client-server interactions**, the solution chosen will depend on the action.

- **For the movement of the wagon**, everything is managed **locally**: in fact, its movement is autonomous, you just need to ensure on the server side that the movement script is run at the same time on all the machines.
- **For player actions** (such as shooting or moving), management is **hybrid**. The action requested by the player is immediate, and then sent to the server for validation. If the current situation of the game matches is what's calculated by the server, then the action is considered valid. If an inconsistency is detected (such as in the case of cheating or network problems), then the player's situation is restored to the last known by the server. This hybrid between local and centralised management ensures that the game runs smoothly while avoiding inconsistencies between machines.
- **Score-related actions** (adding or losing points) are managed **centrally**. The client sends the server any targets that have been eliminated, and the server updates the points if the action is valid. This centralised control prevents cheating if the client is trusted.

B) Key Elements

The key elements of the project are clearly identified, or at least different options are considered.
Our game uses **a game engine** to provide 3D rendering, and to make it easier to manage relationships between elements and between machines.

Interaction with the game is **via the mouse**, which is used to move the camera and click to shoot targets and collect points. **The keyboard** can be used to **move inside the carriage** (the carriage moves automatically), and **to avoid obstacles** (up or down). In the event of a **collision** with another player, they cannot overlap, but each player becomes **translucent**, like a ghost, to make for a more enjoyable game experience. **The players are opponents**, playing 'every man for himself' with the aim of shooting at the target first to deprive their opponents of it. Collisions can be used to push an opponent into an obstacle.

The server enables equipment to be **interfaced centrally**. It would be too risky to operate on a peer-to-peer basis and let the devices communicate freely with each other. As explained in the previous section, the client/server link is a **hybrid** one: the player's movements on the client are executed immediately to ensure a fluid game, but the client sends its actions to the server, which checks them and performs a rollback if any inconsistencies are noticed. As for the counting and display of points, this is completely centralised by the server to avoid any cheating.

A centralised database will store the scores as well as the players' personal information when they join the game. Clients will only have read access, in order to display the score on the screen. **Write access will be managed by the server**: when a player hits a target, the client sends the information back to the server, which checks that it is consistent with the player's last calculated position and the information sent back, and if it is, the score can be updated.

To prevent cheating or data corruption, the communication protocol used implements **encryption and authentication** for each client.

The network aspect is broken down into two parts: **the development of the client-server communication solution** (explained above), and the **management of multiple server-side requests**. Given that each game has a maximum of 4 players, the server must be able to cope with a massive influx of people and generate new games quickly, while ensuring that the different players are synchronised and that client requests are dealt with quickly.

There are two possible solutions for our network infrastructure:

1. Movement management and points management are managed **by the same server** (simplification of the client-server communication solution).

2. One server manages **the generation of games and player movements**, and the other server manages **the points and the database**. This allows our system to withstand a massive influx of players by ensuring that points management is secure, even if movement is disrupted by the influx of players. It also protects the score database in the event of a hack on the game server.

A communications campaign is currently being planned. It will begin with the **creation of a graphic charter**, logos and visuals linked to the game, to create a sense of belonging and a community around Gem Hunter. It will then be centred around various actions on **social networks** and in **the city of Strasbourg**, potentially with posters or visuals to distribute. **The**

alpha version of our game will be open to a panel of external testers, and **the beta version** will be open to a larger number. Feedback will be taken into account to refine the first version of the game.

We decided to adapt our heavy client **to the Web format**. This will **preserve the game's ergonomics** and **mechanics**, while making it accessible to a larger number of players. Our game mechanics are based on fast camera movements and precision, which is complicated to implement on a mobile port. Time permitting, we **may** still **attempt a mobile version** as well.

The architecture could be represented that way:

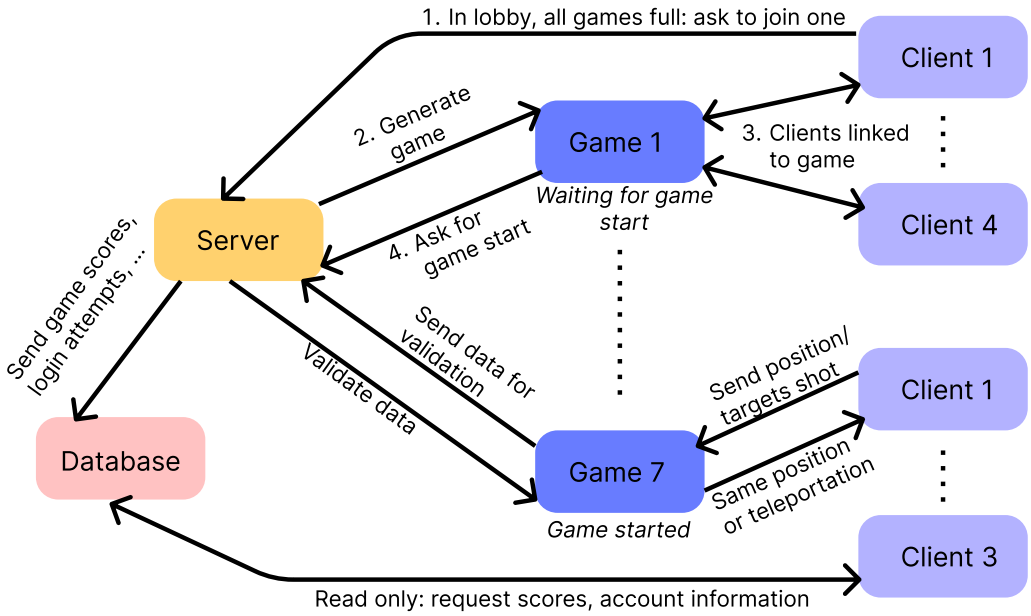


Figure 5: Game management

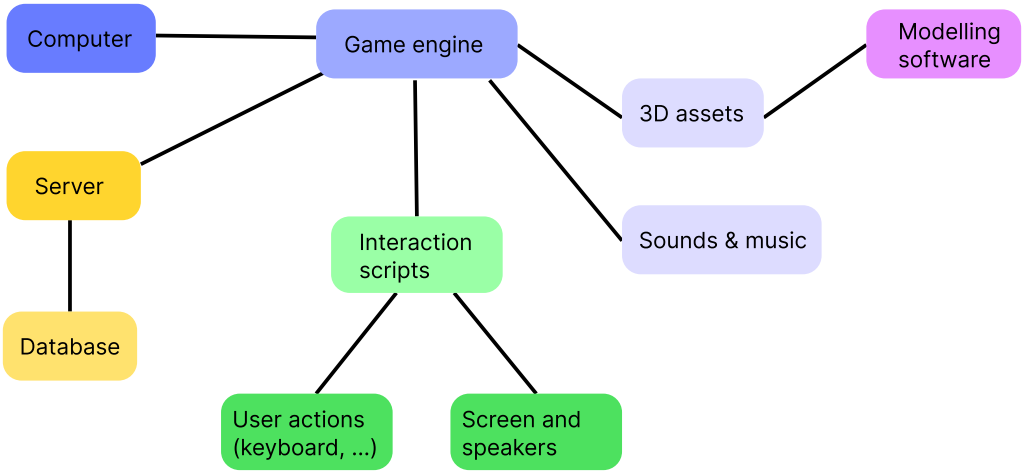


Figure 6: Infrastructure

II. Technologies

The technologies used are virtually all defined.

- For the **modelling part**, we're using **Blender** to create the assets (scenery, players, targets, objects). We've chosen Blender because it's the **favourite solution of most of the people** in the *Modelling* group, being the one used at university. What's more, it's the **leading software** in this field for simple models, with other solutions such as SketchUp and Fusion360 offering features that are unnecessary for our project.

- For the **3D engine and the game logic**, this is a crucial point, as there are major differences in terms of functionality and philosophy between the different game engines. The link between the different parts (game logic, 3D models, network, DB) depends largely on the game engine chosen. We chose **Unity** because it's a **very complete solution** that's **used in industry**. What's more, everyone in the *Game Logic* team has **already used this software** and voted in favour of it. The alternative being considered, Unreal Engine, is more comprehensive and makes it easier to create complex games. However, it's a fairly heavy client, and it's harder to get to grips with if you're used to Unity. The resulting game is also sometimes heavier than if it had been programmed using Unity. **Given the scale of our project**, we preferred Unity.

- The **network part** is centred around **Microsoft's .NET** framework. This framework is **widely used** in video games and other sectors in general. The alternatives, such as Mono, Ruby on Rails and Delphi, are generally less complete and less documented than the .NET framework. What's more, two people in the group **have already used this technology**. The other server-side technologies have not yet been defined and will depend on what is pre-installed on the servers made available.

- The parties are networked by **Mirror Networking**. This is a **Unity-based solution** that allows different clients to be connected in the same lobby. However, it has **technical limitations** (maximum 200 people per session) and a slightly higher latency than its competitors. Despite this, its **integration with Unity is the simplest**, and there is **plenty of documentation**. Synchronisation of parties is handled by the framework. The main alternative for Unity, Fish-Net, is more complete and offers lower latency, but requires more customisation and configuration for features we probably won't be using. Mirror Networking is based on UDP (to avoid the cumbersome TCP header) while implementing flow control and security mechanisms within the UDP packet. Another alternative that was considered at the outset was the **Unity Transport Package**, but this is a solution that is **being used less and less** and has been overtaken by unofficial alternatives.

- The **database** will be a relational database in **SQL**, with a **MySQL database management system** (DBMS). This solution has been chosen because it is the **most widely used DBMS**, and offers enough functionality for the basic use we will be making of it. It is used almost exclusively for connecting players and managing scores. Its design is **close to Oracle**, which is the DBMS we are all trained on. PostgreSQL was considered, but does not necessarily offer any added value for our simple DB.

- Communication between the **DB and the clients** takes place via **WebSocket** because of the persistent connection between the server and the client that it allow. This protocol, based on TCP, is derived from an **international standard** (RFC6455) and is well documented and in use. It is the **most efficient way of communicating**, while remaining **simple to use**. For adaptation to the **Web format**, it is a protocol **supported by the vast majority**

of browsers. It also offers **secure functions**. The other option being considered, i.e. the creation of a dedicated API, requires more configuration for a less satisfactory result.

- Finally, **adaptation to Web format** is facilitated by **Unity**, which makes it easy to export the game. The most suitable technology seems to be **WebGL**, which is **based on JavaScript** and enables 2D and 3D rendering on all browsers. Although this is obviously less efficient than local rendering, it's a simple way of adapting our game. Given that this adaptation phase is coming towards the end of the project, it's **still subject to change**.

The technologies used for **project management** are also clear:

- For day-to-day **task management** and **code version management**, we use **GitLab**. The advantage of this tool is that we can **create issues**, which means we can monitor tasks more closely. It's also a tool that we've been using since we started at university, so we're very familiar with it. The main alternative, GitHub, is obviously quite similar but we're less familiar with it.

- **Team members' hours** are tracked using **Google Sheets**, linked to a Google Drive folder where team members can store their resources. This is a tool that allows **fine-grained control of access** (access to other members' timesheets is forbidden) and easy management of total hours. The other free alternatives all offer fewer features.

- **Daily communication** is via **Discord**, which is used by everyone and is common in university projects. This is the preferred solution for team members, and has therefore been chosen. In an emergency, it is **possible to communicate by telephone**.

- The **Gantt chart**, created with **GanttProject**, is used to manage the progress of the project. Numerous free alternatives exist, but they are almost all less comprehensive than GanttProject, which offers a **free cloud**, **sophisticated ergonomics** and a great deal of **customisation**.

III. Role of the Project Manager

Our general view of the project manager's role is that of coordinator, planner and mediator. The following tasks are identified:

- **Functional description of tasks**: the project manager ensures that the project is broken down into precise functional tasks, and writes a description of what is expected. As the team members are the experts in their field, they take care of the technical description of the task as well as estimating its duration.

- **Evolution of task allocation**: the project manager manages the overall progress of the project, and ensures the overall anticipation of tasks so that the project is finished on time. Using the functional description of the tasks and their estimated time, he updates the Gantt chart and ensures that the milestones set are met. He then allocates the tasks to the development teams responsible, who share out the work according to their affinities and availability. He also manages the monitoring tools, such as Discord and GitLab. Once the task has been completed, it can be validated by the other members of the development team and then implemented.

- **Human relations**: in the event of interpersonal problems within the team, the project manager ensures that the working atmosphere is healthy and does his utmost to resolve conflicts. If this doesn't work, he reports the problem to the tutor. Team members undertake to report any human problems they encounter.

- **Keeping track of hours and availability**: team members undertake to fill in the time tracking sheet honestly and accurately. The project manager monitors the members' sheets and ensures that they are consistent with the justifications. In the event of an abnormal gap or work overload, he will discuss the matter with the person concerned to ensure that the distribution of work is appropriate, and if necessary re-evaluate the distribution of tasks. In the event of unavailability, team members must inform the project manager. Similarly, if the project manager is unavailable, he will give notice and the second in command will take over.

- **Organising team meetings**: the project manager is responsible for organising team meetings. Before the meeting, he sends out the agenda, or if there is no agenda, explains the purpose of the meeting (if the meeting is dedicated to a specific topic), and takes care of the moderation and note-taking. No later than one week after the meeting, minutes are sent out to ensure that what is said is consistent with what was said during the meeting. Meetings can be held on Mondays or at other times. In addition, development team meetings can be held if necessary.

- **Individual monitoring of team members**: every week, if possible, the project manager has a quick meeting with each member of the team to review their work. This gives a clearer picture of how the project is progressing, avoids unnecessary bottlenecks and enables the allocation of tasks to be updated.

- **Link with the tutor**: he is the only person who interacts directly with the tutor. He must therefore collect questions from the team and pass them on, as well as disseminating general information given by the tutor. He undertakes not to misrepresent what the team has to say, as they trust him to represent their interests.

- **The project manager's job is not to code**, but rather to help out if it's within his or her remit. This allows them to take a step back from the code, and therefore to unblock

situations that may be caused by the choice of an unsuitable data structure or algorithm. This position also makes it possible to make the link with the solution developed by another team, and to anticipate blockages.

The project manager is responsible for summarising these principles in an **internal organisation charter**, which will be discussed and validated by the whole team each time it is updated.

IV. Action plan

A) Development Teams

What is special about our team is that several people are multidisciplinary and therefore relevant to several subjects.

That's why we have **several development teams**, and why **each person** can be a **member of several development teams**. The workload per team is therefore not equitable.
Regarding the distribution of work: the project manager **assigns a task to a development team**, which then **distributes the task internally** according to :

1. the skills and interests of each member,
2. the number of tasks each member has completed in the other teams to which they belong.

Once the task has been completed by the member in charge, the other team members are responsible for validating the code before it goes into production.

The development teams are as follows:

- **Modelling.** Responsible for modelling assets (scenery, characters, objects, etc.) and any animations.
- **Game logic.** Responsible for implementing the game algorithms in the game engine and managing the interactions between elements (player, targets, obstacles, etc.), as well as managing the man-machine interface, menus, etc. This team prepares the data to be sent to the network team.
- **Adaptation.** This team manages the adaptation of the game to the Web version and ensures that the game's functionality is preserved. It is a task force made up of members of the game logic team and the network team.
- **Network and server.** Is responsible for receiving and securing the data sent by the game logic team using appropriate protocols. It then validates this data, synchronises it and liaises with the DB team. It also manages the configuration of the server(s), their security, and their resilience to a surge in load.
- **Databases.** This team manages access to the database, securing the data and sending it to clients for display.

The allocation is the following:

- **Project management:** Maxime Zingraff
- **Modelling:** Ugo Berton *(lead)*, Ahmet Berk Ates, Solène Besancon, Anna Callet *(support)*
- **Game logic:** Solène Besancon *(lead)*, Anna Callet, Ahmet Berk Ates, Ugo Berton, Louise-Anne Corbé *(support)*, Hassan El Sahily *(support)*
- **Network and server:** Louise-Anne Corbé **(lead)**, Hassan El Sahily, Solène Besancon *(support)*, Maxime Zingraff *(adviser)*

- **Database:** Hassan El Sahily *(lead)*, Ahmet Berk Ates, Louise-Anne Corbé *(support)*, Maxime Zingraff *(adviser)*
- **Web adaptation:** Ahmet Berk Ates *(lead)*, Solène Besancon *(lead)*, *and any other member depending on their past and current workload when the adaptation will begin*

B) Project Tasks

Several tasks have been identified :

Modelling	Game logic	Network and server	Database	Web adaptation
Preparation	Preparation	Preparation	Preparation	Preparation
Drawing up the graphic charter	Creation of a test scene with car, player and targets	Implementing Mirror on the server	Mirror/ DB links (save points)	Web adaptation
Level design	Creation of basic player movements	Preparing the server (.NET, remote access, etc.)	MySQL DB setup (tables, authentication, triggers)	
Asset modelling	Interaction between the player and the targets	Synchronisation of game launch	Performance evaluation	
Animations and visual effects (characters and targets)	Interaction between players in the wagon (movement, collisions, ghost mode)	Assignment management and generation of parallel games		
Scene dressing (lights, etc.)	Interaction between players and obstacles	Server overload tests		
Creation of sound effects and music	Implementation of Mirror in Unity (Sending movement data to the server; Sending information about projectiles and targets; Query points	Performance evaluation		
	UI creation: score, timer, etc.			
	End of game and victory management			
	Lobby management			
	Menu creation			

We also identified some extra tasks, shared between all teams:

- Communication deployment
  - Campaign planning
  - Preparation of communication material (digital and physical)
  - Publication and printing
- Preparation of documents
- Preparation of the presentation
- Alpha version deployment (restricted panel)
- Beta version deployment (public)

The links between tasks and their duration are shown on the Gantt chart.

We have identified several milestones for this project.

1. **24/03/2025:** First visuals. By this date, it will be possible to have a visual overview of the project and try out the basic functionalities.
2. **07/04 to 14/04/2025:** deployment of the alpha version. This will provide a first playable version of the game, without all its features and perhaps with some bugs.
3. **23/04/2025:** deployment of the beta version. This version will be available to the public and should include most of the features. As soon as the beta version is launched, the communication campaign can begin.
4. **13/05/2025:** release of the definitive version of the game, following feedback from the beta version.

C) Gantt Chart

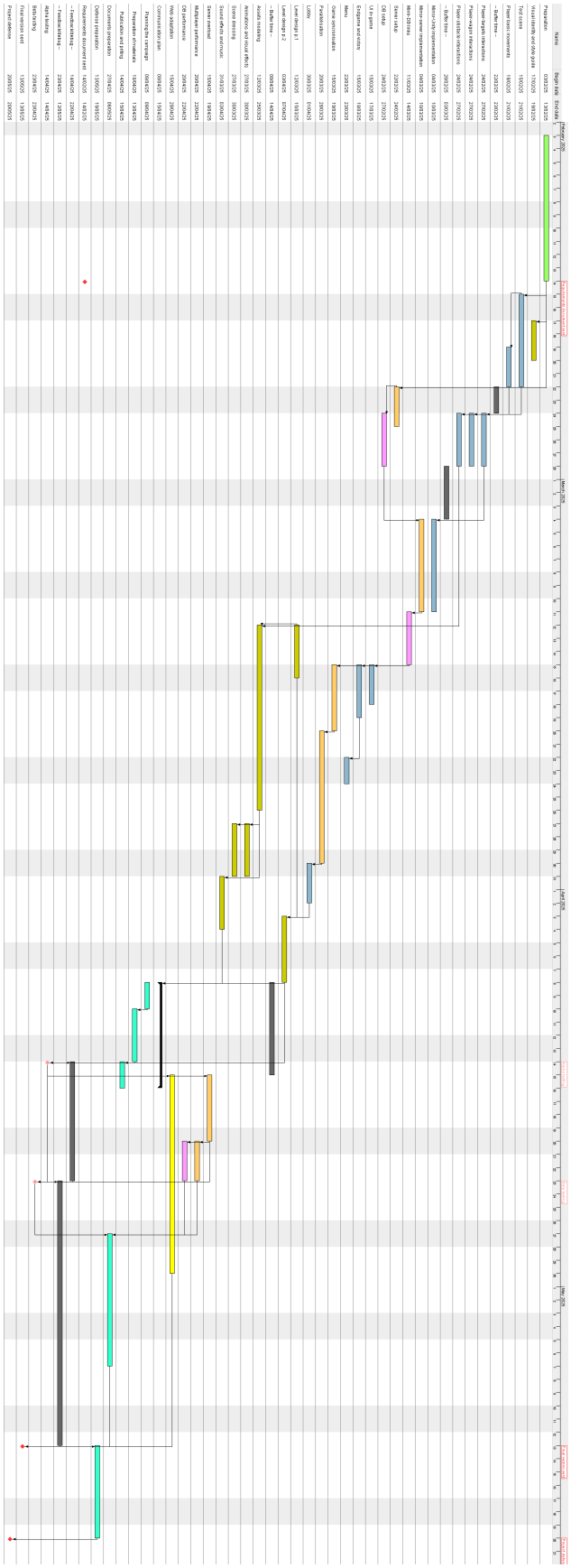


Figure 7: Gantt chart

Legend:

- Green: preparation
- Ochre: modelisation
- Dark blue: game logic
- Orange: network and server
- Pink: database
- Black: buffer time / feedback and debugging
- Yellow Web implementation
- Light red: internal milestones
- Dark red: product owner deadlines

A high-quality version of the Gantt chart can be found on https://imgur.com/a/G6wPIPb/